BINV2150-A — Javascript : avancé

Séance 15 – Fetch, Promesses et useEffect

Haute École Léonard de Vinci

Fetch

- La fonction `fetch` permet d'effectuer une requête HTTP
- Elle prend en argument :
  1. L'URL à laquelle effectuer la requête
  2. Des options sur la requête (méthode, body, headers, ...)

```
fetch('http://localhost:3000/pizzas');
```

```
fetch('http://localhost:3000/pizzas', {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(newPizza),
});
```

- Fetch renvoie une *promesse*

Promesse

- Valeur de retour d'une *fonction asynchrone*
- Fonction asynchrone :
  - Effectue une opération longue (db, réseau, fichier, temps, ...)
  - Ne bloque pas le programme
  - S'exécute en arrière-plan
- 3 états :
  - *Pending* : opération en cours
  - *Fulfilled* : opération réussie, résultat disponible
  - *Rejected* : opération échouée, erreur disponible

Promesse

- Type : `Promise<T>`, avec `T` le type de retour en cas de succès
- Traitée par enregistrement de fonctions callback :
  - `.then()` : fonction appelée en cas de succès
  - `.catch()` : fonction appelée en cas d'erreur
  - Valeur de retour (ou exception) donné en argument de la callback
  - Possible de chaîner les promesses !

```
fetch('https://api.example.com/data')
  .then((response) ⇒ response.json())
  .then((data) ⇒ console.log(data))
  .catch((error) ⇒ console.error('Erreur:', error));
```

```
const promise1 = fetch('https://api.example.com/data');
const promise2 = promise1.then((response) ⇒ response.json());
promise2.then((data) ⇒ console.log(data));
promise2.catch((error) ⇒ console.error('Erreur:', error));
```

Fetch

- Type de retour : `Promise<Response>`
  - `response.status : number` : code de réponse HTTP
  - `response.ok : boolean` : code de réponse 2XX
  - `response.text() : Promise<string>` : contenu du body en string
  - `response.json() : Promise<any>` : contenu json décodé

```
fetch("http://localhost:3000/pizzas")
    .then((response) ⇒ {
        if (!response.ok) throw new Error(`fetch error : ${response.status} : ${response.statusText}`);
        return response.json();
    })
    .then((pizzas: Pizza[]) ⇒ setPizzas(pizzas))
    .catch((err) ⇒ console.error("HomePage::error: ", err));
```

useEffect

- Où faire appel à `fetch` ?
- Dans un gestionnaire d'évennement ✅
- Dans la fonction composant ❌
  - Fonction composant appelée à chaque re-rendu
  - On y retrouve que :
    - Appel à des hooks
    - Définition de variables d'état
    - Définition de fonctions
    - Définition de l'interface JSX
- ➡️ Hook `useEffect`

useEffect

- Exécute du code à des moments contrôllés du cycle de vie du composant
- Pour tout le code à *effets de bord* :
  - **Définition de timers**
- Prend en argument :
  1. La fonction à exécuter
     - **Cette fonction peut renvoyer une fonction à exécuter quand le composant est détaché de l'affichage, s'il faut cleanup des effets de bords**
  2. Un tableau de dépendances
     - Variables d'état
     - Props & Contextes
     - Variables dérivées
- La fonction est exécutée :
  1. Après le premier rendu du composant
  2. Chaque fois qu'une dépendance change

Exemple complet

```tsx
const Menu = () => {
    const [pizzas, setPizzas] = useState<Pizza[]>([]);

    useEffect(() => {
        fetch("http://localhost:3000/pizzas")
            .then((response) => {
                if (!response.ok) throw new Error(`fetch error : ${response.status} : ${response.statusText}`);
                return response.json();
            })
            .then((pizzas: Pizza[]) => setPizzas(pizzas))
            .catch((err) => console.error("HomePage::error: ", err));
    }, []);

    if (!pizzas) return <div>Loading!</div>

    return <>{pizzas.map((pizza) => <PizzaCard pizza={pizza} />)}</>;
}
```